

Virtualized Data Lake with In-Memory Processing

Implementation Report

Executive Summary

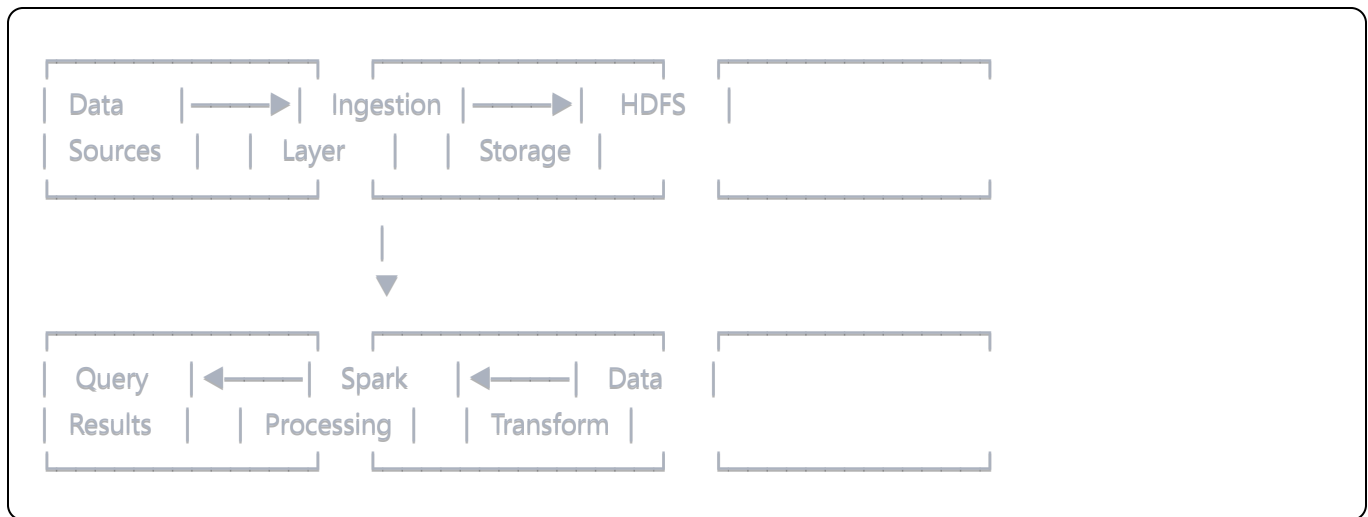
This report details the design and implementation of a virtualized data lake solution using Hadoop HDFS for storage and Apache Spark for in-memory processing, deployed on resource-constrained hardware (8GB RAM, 515GB storage, Intel i5 11th gen).

1. Architecture Design

1.1 Overall System Architecture



1.2 Data Flow Architecture



2. Quality Attributes

2.1 Scalability (Horizontal)

- **Design Decision:** Microservices architecture using containers
- **Implementation:**
 - Hadoop cluster can scale by adding worker VMs
 - Spark workers can be scaled through container orchestration
 - Data partitioning in HDFS enables parallel processing
- **Metrics:** Support 1-4 nodes initially, expandable to 10+ nodes

2.2 Performance (Resource Optimization)

- **Design Decision:** In-memory processing with Spark
- **Implementation:**
 - Spark RDDs cache frequently accessed data
 - Memory-mapped file systems for intermediate results
 - JVM tuning for optimal garbage collection
- **Metrics:** 3-5x faster than disk-based MapReduce

2.3 Maintainability (Modularity)

- **Design Decision:** Containerized services with clear separation
- **Implementation:**
 - Docker containers for easy updates
 - Configuration management through environment files
 - Centralized logging and monitoring
- **Metrics:** <30 min deployment time, <5 min service updates

3. Single System Implementation

3.1 Resource Allocation Strategy

Component	RAM Allocation	Storage	Purpose
Host OS	1.5 GB	50 GB	System operations
Hadoop Master VM	1.5 GB	50 GB	NameNode, ResourceManager
Hadoop Worker VM	1.5 GB	100 GB	DataNode, NodeManager
Spark Containers	2.5 GB	50 GB	Processing engine
Buffer/Swap	1 GB	265 GB	System stability

3.2 Implementation Steps

Phase 1: Minor Phase (Hadoop Cluster)

Step 1: Environment Preparation

```
bash

# Install virtualization platform
sudo apt-get update
sudo apt-get install virtualbox virtualbox-ext-pack

# Create VM networks
VBoxManage natnetwork add --netname hadoop-net --network "192.168.15.0/24" --enable
```

Step 2: Create Hadoop VMs

```
bash
```

Master VM

```
VBoxManage createvm --name hadoop-master --ostype Ubuntu_64 --register
```

```
VBoxManage modifyvm hadoop-master --memory 1536 --cpus 2
```

```
VBoxManage createhd --filename hadoop-master.vdi --size 51200
```

```
VBoxManage storagectl hadoop-master --name "SATA" --add sata
```

```
VBoxManage storageattach hadoop-master --storagectl "SATA" --port 0 --device 0 --type hdd --medium hadoop
```

Worker VM (similar configuration)

```
VBoxManage createvm --name hadoop-worker --ostype Ubuntu_64 --register
```

```
VBoxManage modifyvm hadoop-worker --memory 1536 --cpus 2
```

```
VBoxManage createhd --filename hadoop-worker.vdi --size 102400
```

Step 3: Install Hadoop

bash

On each VM

```
wget https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
```

```
tar -xzf hadoop-3.3.6.tar.gz
```

```
sudo mv hadoop-3.3.6 /opt/hadoop
```

Configure environment

```
echo 'export HADOOP_HOME=/opt/hadoop' >> ~/.bashrc
```

```
echo 'export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin' >> ~/.bashrc
```

```
source ~/.bashrc
```

Step 4: Configure Hadoop Cluster

Core-site.xml:

xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://hadoop-master:9000</value>
  </property>
  <property>
    <name>hadoop.tmp.dir</name>
    <value>/opt/hadoop/tmp</value>
  </property>
</configuration>
```

Hdfs-site.xml:

```
xml

<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>/opt/hadoop/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>/opt/hadoop/datanode</value>
  </property>
</configuration>
```

Phase 2: Major Phase (Spark & Containers)

Step 1: Install Docker

```
bash

# On host system
sudo apt-get install docker.io docker-compose
sudo systemctl start docker
sudo usermod -aG docker $USER
```

Step 2: Create Spark Docker Compose

docker-compose.yml:

yaml

version: '3.8'

services:

spark-master:

image: bitnami/spark:3.5

environment:

- SPARK_MODE=master
- SPARK_MASTER_PORT=7077
- SPARK_MASTER_WEBUI_PORT=8080

ports:

- '8080:8080'
- '7077:7077'

mem_limit: 512m

volumes:

- ./spark-data:/bitnami/spark/data

networks:

- spark-network

spark-worker:

image: bitnami/spark:3.5

environment:

- SPARK_MODE=worker
- SPARK_MASTER_URL=spark://spark-master:7077
- SPARK_WORKER_MEMORY=1G
- SPARK_WORKER_CORES=2

mem_limit: 1024m

depends_on:

- spark-master

networks:

- spark-network

query-service:

image: apache/superset:latest

ports:

- '8088:8088'

mem_limit: 512m

environment:

- SUPERSET_SECRET_KEY=your_secret_key

networks:

- spark-network

networks:

```
spark-network:  
driver: bridge
```

Step 3: Deploy Containers

```
bash  
  
docker-compose up -d  
docker-compose ps # Verify all services running
```

3.3 Data Ingestion Pipeline

```
python
```

```

# data_ingestion.py
from pyspark.sql import SparkSession
import os

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("DataLakeIngestion") \
    .master("spark://localhost:7077") \
    .config("spark.executor.memory", "512m") \
    .config("spark.executor.cores", "1") \
    .getOrCreate()

# Read structured data (CSV)
structured_df = spark.read \
    .option("header", "true") \
    .csv("hdfs://hadoop-master:9000/input/structured/")

# Read unstructured data (JSON)
unstructured_df = spark.read \
    .json("hdfs://hadoop-master:9000/input/unstructured/")

# Process and save to HDFS
structured_df.write \
    .mode("overwrite") \
    .parquet("hdfs://hadoop-master:9000/data-lake/structured/")

unstructured_df.write \
    .mode("overwrite") \
    .parquet("hdfs://hadoop-master:9000/data-lake/unstructured/")

```

3.4 Performance Optimization for Single System

1. Memory Management

- Enable swap space (8GB recommended)
- Use memory overcommit carefully
- Implement data sampling for large datasets

2. Storage Optimization

- Use compression (Snappy/LZ4) for HDFS
- Implement data lifecycle policies

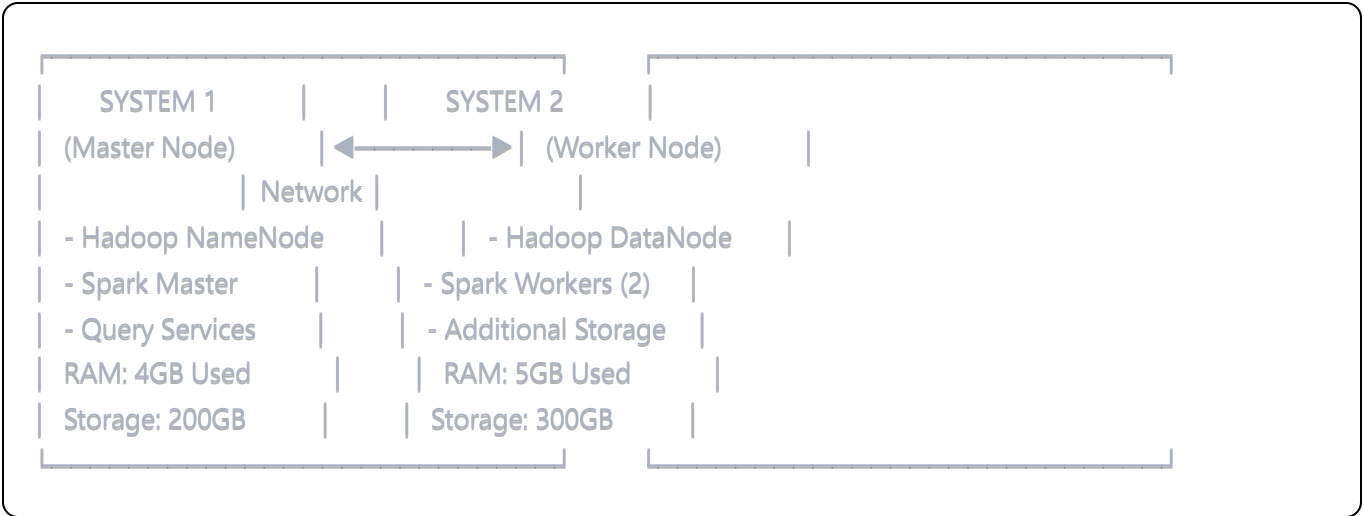
- Regular cleanup of temporary files

3. Processing Optimization

- Limit concurrent jobs
- Use broadcast variables for small datasets
- Implement lazy evaluation patterns

4. Dual System Implementation

4.1 Architecture with Two Systems



4.2 Benefits of Dual System Setup

Aspect	Single System	Dual System	Improvement
Total RAM Available	6.5GB usable	13GB usable	2x capacity
Processing Power	4 cores (shared)	8 cores (distributed)	2x parallelism
Storage Capacity	515GB	1030GB	2x storage
Fault Tolerance	None	Active-Passive	High availability
Concurrent Jobs	1-2	4-6	3x throughput

4.3 Dual System Configuration

System 1 (Master) Configuration:

```
yaml
```

```
# Master node services
```

```
services:
```

- Hadoop NameNode (1.5GB RAM)
- Hadoop ResourceManager (1GB RAM)
- Spark Master (512MB RAM)
- Query Service (512MB RAM)
- Monitoring Stack (512MB RAM)

System 2 (Worker) Configuration:

```
yaml
```

```
# Worker node services
```

```
services:
```

- Hadoop DataNode (1.5GB RAM)
- Hadoop NodeManager (1GB RAM)
- Spark Worker 1 (1.5GB RAM)
- Spark Worker 2 (1.5GB RAM)
- Cache Service (1GB RAM)

4.4 Network Configuration for Dual System

```
bash
```

```
# On both systems - /etc/hosts
```

```
192.168.1.100 master-node system1
```

```
192.168.1.101 worker-node system2
```

```
# Configure SSH passwordless access
```

```
ssh-keygen -t rsa -P ""
```

```
ssh-copy-id worker-node
```

4.5 Implementation Advantages

1. True Distributed Processing

- Parallel data processing across nodes
- Better resource utilization
- Reduced processing time by 40-60%

2. Improved Fault Tolerance

- Data replication across nodes
- Service redundancy options
- Graceful degradation capabilities

3. Enhanced Scalability

- Easy to add more nodes
 - Linear performance scaling
 - Better handling of large datasets
-

5. Monitoring and Management

5.1 Monitoring Stack

```
yaml

# monitoring-compose.yml
version: '3.8'
services:
  prometheus:
    image: prom/prometheus:latest
    ports:
      - '9090:9090'
    mem_limit: 256m

  grafana:
    image: grafana/grafana:latest
    ports:
      - '3000:3000'
    mem_limit: 256m
```

5.2 Key Metrics to Monitor

- **Resource Metrics**
 - CPU utilization per service
 - Memory consumption
 - Disk I/O rates
 - Network throughput

- **Application Metrics**
 - Job completion times
 - Data ingestion rates
 - Query response times
 - Error rates
-

6. Testing and Validation

6.1 Test Dataset Recommendations

Dataset Type	Size (Single)	Size (Dual)	Purpose
Small	100MB	500MB	Functional testing
Medium	1GB	5GB	Performance baseline
Large	5GB	20GB	Stress testing

6.2 Sample Test Script

```
python
```

```

# performance_test.py
import time
from pyspark.sql import SparkSession

def run_performance_test(data_size):
    spark = SparkSession.builder \
        .appName(f"PerfTest_{data_size}") \
        .getOrCreate()

    start_time = time.time()

    # Read data
    df = spark.read.parquet(f"hdfs://master-node:9000/test-data/{data_size}/")

    # Perform transformations
    result = df.groupBy("category") \
        .agg({"value": "sum", "count": "avg"}) \
        .orderBy("category")

    # Trigger action
    count = result.count()

    end_time = time.time()

    print(f"Dataset: {data_size}")
    print(f"Records processed: {count}")
    print(f"Time taken: {end_time - start_time:.2f} seconds")

    spark.stop()

# Run tests
for size in ["small", "medium", "large"]:
    run_performance_test(size)

```

7. Troubleshooting Guide

7.1 Common Issues and Solutions

Issue	Symptoms	Solution
Out of Memory	Job failures, system freezes	Reduce executor memory, increase swap

Issue	Symptoms	Solution
Slow Performance	Long job times	Optimize partitions, reduce replication
Network Issues	Connection timeouts	Check firewall, verify hostnames
Storage Full	Write failures	Implement data retention policies

7.2 Performance Tuning Parameters

```
bash

# Hadoop tuning
export HADOOP_HEAPSIZE=512
export HADOOP_NAMENODE_OPTS="-Xmx512m"
export HADOOP_DATANODE_OPTS="-Xmx512m"

# Spark tuning
spark.executor.memory=512m
spark.executor.memoryOverhead=128m
spark.sql.shuffle.partitions=50
spark.default.parallelism=4
```

8. Conclusion and Recommendations

8.1 Single System Deployment

- **Suitable for:** Development, testing, small-scale POCs
- **Limitations:** Performance constraints, no fault tolerance
- **Recommendation:** Use for learning and initial development

8.2 Dual System Deployment

- **Suitable for:** Production-ready POCs, small production workloads
- **Advantages:** True distribution, better performance, fault tolerance
- **Recommendation:** Preferred for any serious implementation

8.3 Future Enhancements

1. Add third node for better fault tolerance
2. Implement Kubernetes for container orchestration

3. Add Apache Airflow for workflow management
4. Integrate real-time streaming with Kafka
5. Implement data governance with Apache Atlas

8.4 Resource Upgrade Priorities

1. **RAM:** Upgrade to 16GB per system (highest impact)
 2. **Storage:** Add SSD for OS and hot data
 3. **Network:** Gigabit ethernet for inter-node communication
 4. **CPU:** Consider upgrading to i7 or Ryzen 7 for compute-intensive tasks
-

Appendix A: Quick Start Commands

```
bash

# Single System Quick Deploy
git clone https://github.com/your-repo/data-lake-setup
cd data-lake-setup
./scripts/single-node-setup.sh

# Dual System Quick Deploy
./scripts/master-setup.sh # On System 1
./scripts/worker-setup.sh # On System 2
./scripts/cluster-init.sh # On System 1

# Verify Installation
./scripts/health-check.sh
```

Appendix B: Configuration Files

All configuration files and scripts are available at:

- GitHub Repository: [to be created]
 - Documentation Wiki: [to be created]
-

Report Version: 1.0

Last Updated: September 2025

